

COMP40771 — Lab 9

Jupyter Notebook

Lab 9

This jupyter notebook code does the following:

- reads the original Python file
- sends the code to each Ollama model
- stores one improved file per model
- commits and pushes generated files
- supports later testing

In [1]:

```
# CELL 1
import re
import ast
import requests
from pathlib import Path

LOCAL_REPO = Path("/opt/data/comp40771_lab9/lab9")
GITEA_USER = "N1419979"
GITEA_PASSWORD = "f74900a5622a5b2440fbb4e2dc7cae86aa3199d5"
GITEA_REPO = "lab9"
SOURCE_FILE = LOCAL_REPO / "src" / "original_monitor.py"
GITEA_URL = f"http://gitea:3000/{GITEA_USER}/{GITEA_REPO}.git"

OLLAMA_URL = "http://ollama:11434/api/generate"
'''MODELS = [
    "deepcoder:1.5b",
    "qwen2.5-coder:1.5b",
]'''
```

Out [1]:

```
'MODELS = [\n    "deepcoder:1.5b",\n    "qwen2.5-coder:1.5b",\n]'
```

Git tea token

f74900a5622a5b2440fbb4e2dc7cae86aa3199d5

In [2]:

```
# --- Model lists ---
# The lab wants these (they are NOT installed yet on your permanent ollama):
DESIRED_MODELS = ["deepcoder:1.5b", "qwen2.5-coder:1.5b"]

# You already have these installed (from Lab 7):
FALLBACK_MODELS = ["phi3:mini", "llama3.2:1b"]

print("LOCAL_REPO:", LOCAL_REPO)
```

Output:

```
LOCAL_REPO: /opt/data/comp40771_lab9/lab9
```

REMEMBER TO

Update GITEA_USER with your gitea username and GITEA_PASSWORD with your gitea password.

Let's now build the prompt

In [3]:

```
# ==== add at TOP of CELL 2: model planner & background pull ====
import requests, threading

OLLAMA_BASE = "http://ollama:11434"

def list_ollama_models():
    r = requests.get(f"{OLLAMA_BASE}/api/tags", timeout=30)
    r.raise_for_status()
    return [m["name"] for m in r.json().get("models", [])]

def start_background_pull(model: str):
    """Fire-and-forget pull using Ollama HTTP API (no CLI needed)."""
    def _pull():
        try:
            # stream=False is quieter; set True if you want progress lines
            resp = requests.post(
                f"{OLLAMA_BASE}/api/pull",
                json={"model": model, "stream": False},
                timeout=3600
            )
            resp.raise_for_status()
            print(f"[pull] {model} download finished")
        except Exception as e:
            print(f"[pull] {model} pull error: {e}")
    threading.Thread(target=_pull, daemon=True).start()

def plan_models(desired, fallbacks):
    """Return a list to run now; pull missing desired models in background."""
    installed = list_ollama_models()
    run_list = []
    for m in desired:
        if m in installed:
            run_list.append(m)
        else:
            print(f"■ '{m}' not installed → pulling in background; will use a fallback this run.")
            start_background_pull(m)
            fb = next((f for f in fallbacks if f in installed), None)
            if fb:
                run_list.append(fb)
            else:
                print("■■ No fallback installed; this slot will be skipped.")
    return run_list

def call_ollama_with_fallback(model, prompt, url):
    """Wrap your existing query_ollama to classify 404 as 'model missing'."""
    try:
        return query_ollama(model, prompt, url) # uses your existing function
    except requests.HTTPError as e:
        if e.response is not None and e.response.status_code == 404:
            # Let the caller skip this model for this run
            raise RuntimeError(f"Model '{model}' is not installed yet") from e
        raise
```

In [4]:

```
# CELL 2
original_code = SOURCE_FILE.read_text(encoding="utf-8")
print(original_code[:1500])

def build_prompt(code: str) -> str:
    return f"""
    You are improving a Python script used in a CI/CD-ML engineering lab.
```

Goals:

1. Preserve the original functionality exactly:
 - recursively list all files and folders under /opt/data
 - collect memory usage during 1 second
 - print the results to screen
2. Improve code quality, readability, structure, robustness, and comments.
3. Keep the code executable as a standalone Python script.
4. Do not remove any required functionality.
5. Return only valid Python code, with no markdown fences and no explanation.

Original code:

```
{code}
"""
```

```
def query_ollama(model: str, prompt: str, ollama_url: str) -> str:
    response = requests.post(
        ollama_url,
        json={
            "model": model,
            "prompt": prompt,
            "stream": False,
        },
        timeout=300,
    )
    response.raise_for_status()
    data = response.json()
    return data["response"]
```

Output:

```
import os
import time
import tracemalloc
from pathlib import Path
```

```
def list_all_paths(root="/opt/data"):
    root_path = Path(root)
    results = []

    if not root_path.exists():
        print(f"Path does not exist: {root}")
        return results

    for current_root, dirs, files in os.walk(root):
        for d in dirs:
            results.append(os.path.join(current_root, d))
        for f in files:
            results.append(os.path.join(current_root, f))

    return sorted(results)

def collect_memory_usage(duration_seconds=1.0, interval_seconds=0.1):
    tracemalloc.start()
    samples = []

    start = time.time()
    while time.time() - start < duration_seconds:
        current, peak = tracemalloc.get_traced_memory()
        samples.append({
            "timestamp": time.time(),
            "current_bytes": current,
            "peak_bytes": peak,
        })
        time.sleep(interval_seconds)
```

```

current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

return {
    "samples": samples,
    "final_current_bytes": current,
    "final_peak_bytes": peak,
}

def main():
    print("Listing files and folders under /opt/data and subfolders")
    print("-" * 60)
    paths = list_all_paths("/opt/data")

    if not paths:
        print("No files or folders found.")
    else:
        for p in paths:
            print(p)

    print("\nCollecting memory usage for 1 second")
    print("-" * 60)
    memory_info = collect_memory_usage(duration

```

Let's now query the model and validating the code

In [5]:

```

# CELL 2.5 – Model planner
# ==== MODEL PLANNER: use what's present now, pull what's missing in background ====
import requests, threading

OLLAMA_BASE = "http://ollama:11434"

def list_ollama_models():
    r = requests.get(f"{OLLAMA_BASE}/api/tags", timeout=30)
    r.raise_for_status()
    data = r.json()
    return [m["name"] for m in data.get("models", [])]

def start_background_pull(model: str):
    # fire-and-forget pull; prints progress lines if you want to see them
    def _pull():
        try:
            with requests.post(
                f"{OLLAMA_BASE}/api/pull",
                json={"model": model, "stream": True},
                stream=True, timeout=3600
            ) as r:
                for line in r.iter_lines():
                    if line:
                        print(f"[pull:{model}] {line.decode(errors='ignore')[:120]}")
        except Exception as e:
            print(f"[pull:{model}] error: {e}")
    t = threading.Thread(target=_pull, daemon=True)
    t.start()
    return t

def plan_models(desired, fallbacks):
    installed = list_ollama_models()
    run_list = []
    for m in desired:
        if m in installed:
            run_list.append(m)

```

```

else:
    print(f"■ '{m}' not installed → pulling in background, will use a fallback this run.")
    start_background_pull(m)
    fb = next((f for f in fallbacks if f in installed), None)
    if fb:
        run_list.append(fb)
    else:
        print("■■ No fallback installed; this slot will be skipped for now.")
return run_list

```

In [6]:

```

# Plan the models for THIS run (won't block; starts downloads in background)
MODELS_TO_RUN = plan_models(DESIRED_MODELS, FALLBACK_MODELS)
print("Will run now:", MODELS_TO_RUN)

```

Output:

```
Will run now: ['deepcoder:1.5b', 'qwen2.5-coder:1.5b']
```

In [7]:

```
# CELL 3
```

```

def is_valid_python(code_text: str) -> bool:
    print("Checking if extracted text is valid Python...")

    if not code_text or not code_text.strip():
        print("Validation failed: empty code block")
        return False

    bad_markers = ["<think>", "</think>", "<justification>", "</justification>"]

    if any(marker in code_text for marker in bad_markers):
        print("Validation failed: reasoning markers detected inside code")
        return False

    try:
        ast.parse(code_text)
        print("Python syntax validation: SUCCESS")
        return True
    except SyntaxError as e:
        print("Python syntax validation: FAILED")
        print("Syntax error:", e)
        return False

def extract_fenced_python(raw_text: str):
    print("Attempting to extract fenced code blocks...")

    patterns = [
        r"```python\s*(.*?)\s*```",
        r"```py\s*(.*?)\s*```",
        r"```[a-zA-Z0-9_-]*\s*\n(.*?)\s*```",
        r"```\s*(.*?)\s*```",
    ]

    matches = []
    seen = set()

    for pattern in patterns:
        found = re.findall(pattern, raw_text, re.DOTALL | re.IGNORECASE)
        for block in found:
            candidate = block.strip()
            if candidate and candidate not in seen:
                matches.append(candidate)

```

```

        seen.add(candidate)

if not matches:
    print("No fenced code blocks found")
    return None

print(f"Found {len(matches)} fenced blocks")

first_candidate = matches[0]

for idx, candidate in enumerate(matches):
    print(f"Validating fenced block {idx + 1}")
    if is_valid_python(candidate):
        print("Valid fenced Python block extracted")
        return candidate

print("No valid fenced Python block found")
print("Returning first fenced block for inspection")
return first_candidate

def extract_plain_python(raw_text: str):
    print("Attempting plain-text Python extraction...")

    text = raw_text.strip()

    if is_valid_python(text):
        print("Entire response is valid Python")
        return text

    print("Scanning lines for Python starting points...")

    lines = text.splitlines()

    python_starters = (
        "import ",
        "from ",
        "def ",
        "class ",
        "if __name__",
        "for ",
        "while ",
        "try:",
        "with ",
        "print(",
        "@",
    )

    start_idx = None

    for i, line in enumerate(lines):
        stripped = line.strip()
        if stripped.startswith(python_starters):
            start_idx = i
            print(f"Possible Python start detected at line {i}")
            break

    if start_idx is not None:
        candidate = "\n".join(lines[start_idx:]).strip()

        if is_valid_python(candidate):
            print("Plain Python extraction successful")
            return candidate

    print("Plain-text candidate extracted but is not valid Python")

```

```

        return candidate

    print("Plain extraction failed")
    return None

def extract_sections(raw_text: str):
    print("Extracting reasoning sections and code...")

    think_match = re.search(
        r"<think>\s*(.*?)\s*</think>",
        raw_text,
        re.DOTALL | re.IGNORECASE,
    )
    justification_match = re.search(
        r"<justification>\s*(.*?)\s*</justification>",
        raw_text,
        re.DOTALL | re.IGNORECASE,
    )

    think_text = think_match.group(1).strip() if think_match else ""
    justification_text = justification_match.group(1).strip() if justification_match else ""

    code_text = extract_fenced_python(raw_text)

    if code_text is None:
        print("Trying plain extraction...")
        code_text = extract_plain_python(raw_text)

    explanation_parts = []

    if think_text:
        explanation_parts.append("THINK\n" + "-" * 80 + f"\n{think_text}")

    if justification_text:
        explanation_parts.append("JUSTIFICATION\n" + "-" * 80 + f"\n{justification_text}")

    if not explanation_parts:
        explanation_parts.append("RAW RESPONSE\n" + "-" * 80 + f"\n{raw_text}")

    explanation_text = "\n\n".join(explanation_parts)

    return {
        "code": code_text or "",
        "text": explanation_text,
        "raw": raw_text,
        "valid_python": is_valid_python(code_text or ""),
    }

print("\nStarting LLM code generation pipeline")
print("=" * 80)

generated_files = []
generated_text_files = []
failed_models = []

for model in MODELS_TO_RUN: #Change for model in MODELS: → for model in MODELS_TO_RUN:
    print("\n")
    print("=" * 80)
    print(f"Processing model: {model}")
    print("=" * 80)

    safe_name = model.replace(":", "_").replace(".", "_").replace("-", "_")
    py_file = LOCAL_REPO / "src" / f"improved_{safe_name}.py"

```

```

txt_file = LOCAL_REPO / "src" / f"improved_{safe_name}.txt"

prompt = build_prompt(original_code)

print("Sending prompt to Ollama...")

#Instead of:
#raw_response = query_ollama(model, prompt, OLLAMA_URL)

# wrap only that one call to query_ollama(...) with a small try/except.
try:
    raw_response = call_ollama_with_fallback(model, prompt, OLLAMA_URL)
except RuntimeError as e:
    # Model missing this run-background pull already started by the planner.
    print(e)
    failed_models.append(model)
    continue

print("Response received")
print(f"Response length: {len(raw_response)} characters")

print("\nExtracting sections...")
parsed = extract_sections(raw_response)

code_text = parsed["code"]
valid_python = parsed["valid_python"]

if not code_text.strip():
    print("No code could be extracted for model:", model)

    failed_models.append(model)

    txt_file.write_text(
        f"Model: {model}\n"
        f"No code could be extracted.\n\n"
        f"RAW RESPONSE\n{'-' * 80}\n{raw_response}\n",
        encoding="utf-8",
    )

    print("Raw response saved to:", txt_file)
    continue

print("Writing extracted code to file:", py_file)
py_file.write_text(code_text.rstrip() + "\n", encoding="utf-8")

validation_header = (
    f"Model: {model}\n"
    f"Valid Python: {valid_python}\n\n"
)

txt_file.write_text(
    validation_header + parsed["text"] + "\n\nEXTRACTED CODE\n" + "-" * 80 + f"\n{code_text}\n",
    encoding="utf-8",
)

generated_text_files.append(txt_file)

if valid_python:
    generated_files.append(py_file)
    print("Files generated successfully")
else:
    failed_models.append(model)
    print("Extracted code was saved, but it is not valid Python")

print("\nCode preview:")

```



```

    print("-" * 60)
    print(code_text[:500])

print("\nPipeline finished")
print("=" * 80)

print("\nGenerated Python files:")
for path in generated_files:
    print(path)

print("\nGenerated explanation files:")
for path in generated_text_files:
    print(path)

print("\nFailed models:")
print(failed_models)

```

Output:

```

Starting LLM code generation pipeline
=====

Processing model: deepcoder:1.5b
=====
Sending prompt to Ollama...
Response received
Response length: 6073 characters

Extracting sections...
Extracting reasoning sections and code...
Attempting to extract fenced code blocks...
Found 3 fenced blocks
Validating fenced block 1
Checking if extracted text is valid Python...
Python syntax validation: SUCCESS
Valid fenced Python block extracted
Checking if extracted text is valid Python...
Python syntax validation: SUCCESS
Writing extracted code to file: /opt/data/comp40771_lab9/lab9/src/improved_deepcoder_1_5b.py
Files generated successfully

```

Code preview:

```

-----
import os
import time
import tracemalloc
from pathlib import Path

def list_all_paths(root="/opt/data"):
    """List all files and folders under the specified root path."""
    paths = []
    try:
        current_root = str(root)
    except TypeError:
        return [] # Return empty if root is not a string
    for root, dirs, files in os.walk(current_root):
        for d in dirs:
            paths.append(str(d))
        for f in files:
            paths.append(str(f))
    return sorted(list(se

```

Processing model: qwen2.5-coder:1.5b

=====

Sending prompt to Ollama...

Response received

Response length: 1962 characters

Extracting sections...

Extracting reasoning sections and code...

Attempting to extract fenced code blocks...

Found 3 fenced blocks

Validating fenced block 1

Checking if extracted text is valid Python...

Python syntax validation: SUCCESS

Valid fenced Python block extracted

Checking if extracted text is valid Python...

Python syntax validation: SUCCESS

Writing extracted code to file: /opt/data/comp40771_lab9/lab9/src/improved_qwen2_5_coder_1_5b.py

Files generated successfully

Code preview:

```
import os
```

```
import time
```

```
from pathlib import Path
```

```
def list_all_paths(root="/opt/data"):
```

```
    root_path = Path(root)
```

```
    results = []
```

```
    if not root_path.exists():
```

```
        print(f"Path does not exist: {root}")
```

```
        return results
```

```
    for current_root, dirs, files in os.walk(root):
```

```
        for d in dirs:
```

```
            results.append(os.path.join(current_root, d))
```

```
        for f in files:
```

```
            results.append(os.path.join(current_root, f))
```

```
    return sorted(results)
```

```
def collect_memory_usage
```

Pipeline finished

=====

Generated Python files:

/opt/data/comp40771_lab9/lab9/src/improved_deepcoder_1_5b.py

/opt/data/comp40771_lab9/lab9/src/improved_qwen2_5_coder_1_5b.py

Generated explanation files:

/opt/data/comp40771_lab9/lab9/src/improved_deepcoder_1_5b.txt

/opt/data/comp40771_lab9/lab9/src/improved_qwen2_5_coder_1_5b.txt

Failed models:

[]

In [8]:

```
# what the 'planner' sees
```

```
print("Installed models (Ollama):", list_ollama_models())
```

Output:

Installed models (Ollama): {'deepcoder:1.5b', 'qwen2.5-coder:1.5b', 'phi3:mini', 'llama3.2:1b'}

Let's commit the changes to Gitea

In [9]:

```
# CELL 4
import subprocess

def run_cmd(cmd, cwd=None, check=True):
    result = subprocess.run(
        cmd,
        cwd=cwd,
        text=True,
        capture_output=True
    )

    if check and result.returncode != 0:
        print("Command failed:", " ".join(cmd))
        print("STDOUT:\n", result.stdout)
        print("STDERR:\n", result.stderr)
        raise RuntimeError("Command execution failed")

    print(result.stdout)
    return result

run_cmd(["git", "add", "."], cwd=LOCAL_REPO)
run_cmd(
    ["git", "commit", "-m", "Add LLM-generated improved Python scripts"],
    cwd=LOCAL_REPO,
    check=False
)

auth_url = f"http://{GITEA_USER}:{GITEA_PASSWORD}@gitea:3000/{GITEA_USER}/{GITEA_REPO}.git"
run_cmd(["git", "remote", "set-url", "origin", auth_url], cwd=LOCAL_REPO)

run_cmd(["git", "push", "-u", "origin", "main"], cwd=LOCAL_REPO)
```

Output:

```
[main 0a21b2f] Add LLM-generated improved Python scripts
4 files changed, 84 insertions(+), 82 deletions(-)
```

```
branch 'main' set up to track 'origin/main'.
```

Out [9]:

```
CompletedProcess(args=['git', 'push', '-u', 'origin', 'main'], returncode=0, stdout="branch 'main' set up to t
in/main'.\n", stderr='remote: . Processing 1 references\nremote: Processed 1 references in total
http://gitea:3000/N1419979/lab9.git\n 1c6e781..0a21b2f main -> main\n')
```

BUILD_TOKEN

dad2e6f529d0910e1e40f854353f7393

JENKINS_API_TOKEN

1147f86133e24536af25f7180f1db0f98d

Let's now inform Jenkins that there is a new commit using REST

Jenkins will build and run the tests for the 3 files (the original and the generated using the 2x LLMs). The output of the build will be published in topic jenkins/pipeline/results

In [13]:

```

#CELL 5
import requests

JENKINS_URL = "http://jenkins-comp40771:8080"
JOB_NAME = "lab9"
BUILD_TOKEN = "dad2e6f529d0910e1e40f854353f7393"

JENKINS_USER = "N1419979"
JENKINS_API_TOKEN = "1147f86133e24536af25f7180f1db0f98d"

session = requests.Session()
session.auth = (JENKINS_USER, JENKINS_API_TOKEN)

print("Requesting Jenkins crumb...")
crumb_response = session.get(f"{JENKINS_URL}/crumbIssuer/api/json")
crumb_response.raise_for_status()

crumb_data = crumb_response.json()
crumb_field = crumb_data["crumbRequestField"]
crumb_value = crumb_data["crumb"]

headers = {
    crumb_field: crumb_value
}

build_url = f"{JENKINS_URL}/job/{JOB_NAME}/build?token={BUILD_TOKEN}"

print("Triggering Jenkins pipeline...")
response = session.post(build_url, headers=headers)

if response.status_code in [200, 201, 202]:
    print("Build successfully triggered.")
else:
    print("Failed to trigger build.")
    print("Status:", response.status_code)
    print(response.text)

```

Output:

```

Requesting Jenkins crumb...
Triggering Jenkins pipeline...
Build successfully triggered.

```

Let's now subscribe the topic jenkins/pipeline/results to receive the result of Jenkins

To avoid having to login into Jenkins, we will subscribe the topic jenkins/pipeline/results to inspect the results. The code below runs until a message is published into the topic jenkins/pipeline/results.

In [14]:

```

# install paho-mqtt v2 (your code uses CallbackAPIVersion.VERSION2, which is only in v2+)
%pip install "paho-mqtt>=2.0.0"

```

Output:

```

Requirement already satisfied: paho-mqtt>=2.0.0 in /root/.env/lib/python3.12/site-packages (2.1.0)
Note: you may need to restart the kernel to use updated packages.

```

In [16]:

```

#CELL 6
import json
import time
import paho.mqtt.client as mqtt
BROKER = "mqtt-broker"
PORT = 1883

```


[illegible]

```

=====
Response length: 5720 characters
Saved: /opt/data/comp40771_lab9/lab9-advanced/src/improved_qwen2_5_coder_1_5b.py
Preview:
import random
import time

SCENARIOS = ["MI", "MF", "RF", "TO"]
PROFILES = ["BB", "unsafe", "fragile"]

def simulate_run(profile: str, scenario: str, seed: int):
    """
    Simulate a sandboxed execution run.
    Returns (event_log, features).

    profile : "BB" (benign baseline), "unsafe", or

Generated: ['/opt/data/comp40771_lab9/lab9-advanced/src/improved_qwen2_5_coder_1_5b.py']
Failed:      ['deepcoder:1.5b']

```

Run this single notebook cell exactly as it is. It only targets Deepcoder, waits longer, validates the returned code, and saves the missing file:

In [5]:

```

import re
import ast
import requests
from pathlib import Path

LOCAL_REPO = Path("/opt/data/comp40771_lab9/lab9-advanced")
SOURCE_FILE = LOCAL_REPO / "src" / "original_art11_sim.py"
OLLAMA_URL = "http://ollama:11434/api/generate"

original_code = SOURCE_FILE.read_text(encoding="utf-8")

def build_prompt(code):
    return f"""
You are improving a Python script used in a CI/CD-ML security lab.

Goals:
1. Preserve all original functionality exactly.
2. simulate_run(profile, scenario, seed) must still return (event_log, features).
3. Keep all scenarios: MI, MF, RF, TO.
4. Keep all profiles: BB, unsafe, fragile.
5. Keep all feature keys and event-log fields.
6. Improve readability, structure, robustness, and comments.
7. Keep the script executable as a standalone Python script.
8. Return only valid Python code, with no markdown fences and no explanation.

Original code:
{code}
"""

def is_valid_python(code_text):
    if not code_text or not code_text.strip():
        return False
    try:
        ast.parse(code_text)
        return True
    except SyntaxError:
        return False

def extract_code(raw_text):

```



```

patterns = [
    r"``python\s*(.*?)\s*``",
    r"``py\s*(.*?)\s*``",
    r"``[a-zA-Z0-9_-]*\s*\n(.*?)\s*``",
    r"``\s*(.*?)\s*``",
]
for pattern in patterns:
    matches = re.findall(pattern, raw_text, re.DOTALL | re.IGNORECASE)
    for block in matches:
        candidate = block.strip()
        if is_valid_python(candidate):
            return candidate
text = raw_text.strip()
if is_valid_python(text):
    return text
return None

model = "deepcoder:1.5b"
safe_name = model.replace(":", "_").replace(".", "_").replace("-", "_")
py_file = LOCAL_REPO / "src" / f"improved_{safe_name}.py"
txt_file = LOCAL_REPO / "src" / f"improved_{safe_name}.txt"

prompt = build_prompt(original_code)

response = requests.post(
    OLLAMA_URL,
    json={"model": model, "prompt": prompt, "stream": False},
    timeout=900,
)
response.raise_for_status()

raw = response.json()["response"]
code = extract_code(raw)

if not code:
    txt_file.write_text(raw, encoding="utf-8")
    raise RuntimeError("Deepcoder returned no valid Python code.")

py_file.write_text(code.rstrip() + "\n", encoding="utf-8")
txt_file.write_text(raw, encoding="utf-8")

print(py_file)

```

Output:

```

/opt/data/comp40771_lab9/lab9-advanced/src/improved_deepcoder_1_5b.py

```

Part 6 — Trigger Jenkins via REST from the Notebook

What this part is: Instead of clicking "Build Now" manually, you send an HTTP request from inside the comp40771 notebook to trigger the lab9-advanced pipeline. This is a required deliverable — you must document the endpoint, authentication, and how you confirmed the build.

In [11]:

```

import requests
import time

JENKINS_URL = "http://jenkins-comp40771:8080"
JOB_NAME = "lab9-advanced"
BUILD_TOKEN = "1c24d3825784ad1867f0096ad2a849ed"
JENKINS_USER = "N1419979"
JENKINS_API_TOKEN = "118d04ec92a9718d32c16dde8af9803a6c"
session = requests.Session()

```

**If you want to test the subscriber without waiting for a full Jenkins build

You can also verify it works independently by publishing a test message directly from the notebook:**

In [10]:

```
import json
import paho.mqtt.client as mqtt

client = mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.connect("mqtt-broker", 1883, 60)
client.loop_start()

test_payload = json.dumps({
    "pipeline": "lab9-advanced",
    "job": "lab9-advanced",
    "build": "TEST",
    "host": "http://localhost:8080/",
    "files_checked": ["original_art11_sim.py",
                     "improved_deepcoder_1_5b.py",
                     "improved_qwen2_5_coder_1_5b.py"],
    "overall": "SUCCESS",
    "report_excerpt": "Tests run : 9 | Passed : 9"
})

client.publish("jenkins/pipeline/results", test_payload, qos=1)
client.loop_stop()
client.disconnect()
print("Test message published.")
```

Output:

Test message published.

In []:

— End of Notebook —